

Document AI - provenance, scores et informations exploitables

Synthèse imprimable pour Factory Writer : ce que Google Document AI expose réellement, comment y accéder avec le SDK Python / Document AI Toolbox, et ce qu'il faut stocker pour justifier les règles et les faits extraits.

Besoin	Accès	Comment y accéder	Exemple	Limite
Identifier un chunk	Oui	Document AI expose Document.chunkedDocument.chunks[].chunkId . En Python: chunk.chunk_id . Avec Toolbox: for chunk in wrapped_document.chunks .	docai_chunk_id = chunk-0007	Disponible seulement si le Layout Parser produit des chunks.
Type de source du fragment	Non Google	Champ à créer côté Factory Writer selon le chemin d'extraction: chunk Document AI, fallback texte complet, block layout, etc.	source_type = DOCUMENT_AI_CHUNK	N'existe pas dans Document AI. C'est une donnée applicative.
Texte du chunk	Oui	Document AI expose chunk.content . Le SDK Python et la Toolbox donnent accès au même champ.	contenu = La voix Axolotl privilégie un vocabulaire botanique...	Si aucun chunk n'est produit, fallback possible sur document.text .
Pages du chunk	Oui	Document AI expose chunk.pageSpan.pageStart et chunk.pageSpan.pageEnd . En Python: chunk.page_span.page_start .	page_start = 2, page_end = 3	Donne une plage de pages, pas une zone précise dans la page.
Titres de contexte	Oui	Document AI expose chunk.pageHeaders[] et chunk.pageFooters[] . En Python: chunk.page_headers, chunk.page_footers .	page_headers_json = [Voix de marque, Lexique recommandé]	Ce sont des headers/footers associés, pas forcément tous les titres logiques.
Structure du document	Oui	Document AI expose document.documentLayout.blocks[] . Toolbox expose wrapped_document.document_layout.blocks . Les blocks ont blockId, pageSpan, boundingBox, textBlock/tableBlock/listBlock .	block_type = textBlock, textBlock.type = heading-2	chunk.sourceBlockIds[] est marqué Unused dans la doc: lien chunk -> block non garanti.
Coordonnées visuelles	Oui, indirect	Document AI expose Layout.boundingPoly, PageRef.boundingPoly, DocumentLayoutBlock.boundingBox . En Python: bounding_poly.vertices ou normalized_vertices . Option utile: returnBoundingBoxes=true .	normalizedVertices = [{x:0.12,y:0.30}, ...]	Un chunk n'a pas directement de bounding box. Pour encadrer le PDF, passer par block, entity, paragraph, token ou page anchor.
Confiance OCR/layout	Oui, partiel	Document AI expose Layout.confidence sur plusieurs objets: token, table, paragraph, page element, etc. En Python: page.tokens[].layout.confidence .	extraction_confidence = 0.96	Pas de chunk.confidence . Pour un chunk, calculer un score agrégé côté Factory Writer.
Qualité de page	Oui, sous conditions	Enterprise OCR quality analysis expose pages[].imageQualityScores.qualityScore . En Python: page.documentai_object.image_quality_scores.quality_score . Option OCR: enable_image_quality_scores=true .	qualityScore = 0.91	Avec Layout Parser seul, ce n'est pas garanti. Pour un guardrail fort, prévoir un passage Enterprise OCR qualité.
Défauts image	Oui, sous conditions	Document AI expose pages[].imageQualityScores.detectedDefects[] . En Python: detected_defect.type_ et detected_defect.confidence .	{type: defect_blurry, confidence: 0.74}	Dépend de l'analyse qualité OCR activée.
Entités structurées	Oui	Document AI expose document.entities[] . Toolbox expose wrapped_document.entities et get_entity_by_type(...) .	{type: material, mentionText: teck grade A, confidence: 0.93}	Le Layout Parser ne produit pas forcément des entités métier. Pour dimensions/matériaux: Custom Extractor/Form Parser.
Position exacte d'une entité	Oui	Document AI expose entity.pageAnchor.pageRefs[].boundingPoly . En Python: entity.page_anchor.page_refs[].bounding_poly .	page = 3, boundingPoly = {...}	Certains champs dérivés peuvent ne pas avoir de textAnchor ou pageAnchor .
Valeur normalisée	Oui, selon type	Document AI expose entity.normalizedValue . En Python: entity.normalized_value.float_value, integer_value, date_value, money_value, boolean_value, text .	Source: 2,10 m -> floatValue = 2.10	Pas disponible pour tous les champs. Dépend du processor et du type d'entité.

Besoin	Accès	Comment y accéder	Exemple	Limite
Méthode d'obtention	Oui	Document AI Entity expose entity.method . Enum Python: EXTRACT , DERIVE .	method = EXTRACT	Pour les faits techniques, DERIVE ne doit pas être preuve canonique.
Version du processor	Oui	Utiliser un resource name avec processorVersions/<version> . En Python: processor_version_path(...) . Sinon Document AI utilise la version par défaut.	pretrained-layout-parser-v1.5-2025-08-25	Si la version n'est pas figée ou stockée, la reproductibilité est faible.
Config de requête	Oui	Champs SDK: BatchProcessRequest.process_options , DocumentOutputConfig.GcsOutputConfig.field_mask , sharding_config , LayoutConfig.chunking_config , return_bounding_boxes .	{includeAncestorHeadings: true, returnBoundingBoxes: true}	Google ne persiste pas un snapshot métier pour nous. À stocker côté Factory Writer.
Évaluation modèle	Oui, offline	Document AI Evaluation API / console expose precision, recall, F1 par processor version et label.	precision = 0.93, recall = 0.88, f1 = 0.90	Pas un score par document/règle. C'est une évaluation offline sur dataset annoté.

Recommandation POC

Pour le guide de style, stocker au minimum source_type, docai_chunk_id, contenu, page_start, page_end, page_headers_json et provenance_status dans fragment_style. Pour les dossiers techniques usine, utiliser des entités structurées avec mentionText, normalizedValue, confidence, pageAnchor et boundingPoly ; bloquer la publication si une dimension ou un matériau n'a pas de preuve documentaire vérifiable.

Sources vérifiées

- Document AI Layout Parser
- Document schema REST v1
- Document AI RPC reference
- ProcessOptions
- DocumentOutputConfig
- Send a processing request
- Document AI Toolbox
- Document AI Toolbox Python Document
- Enterprise Document OCR quality analysis
- Custom Extractor with GenAI
- Evaluate processor performance